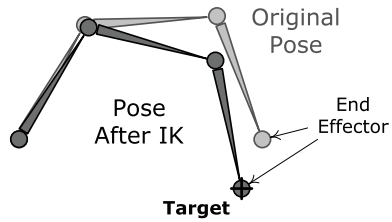

Figure 12.44. Inverse kinematics attempts to bring an end effector joint into a target global pose by minimizing the error between them.

this happen.

A regular animation clip is an example of *forward kinematics* (FK). In forward kinematics, the input is a set of local joint poses, and the output is a global pose and a skinning matrix for each joint. Inverse kinematics goes in the other direction: The input is the desired global pose of a single joint, which is known as the *end effector*. We solve for the *local* poses of other joints in the skeleton that will bring the end effector to the desired location.

Mathematically, IK boils down to an *error minimization* problem. As with most minimization problems, there might be one solution, many or none at all. This makes intuitive sense: If I try to reach a doorknob that is on the other side of the room, I won't be able to reach it without walking over to it. IK works best when the skeleton starts out in a pose that is reasonably close to the desired target. This helps the algorithm to focus in on the "closest" solution and to do so in a reasonable amount of processing time. Figure 12.44 shows IK in action.

Imagine a two-joint skeleton, each of which can rotate only about a single axis. The rotation of these two joints can be described by a two-dimensional angle vector $\theta = \begin{bmatrix}\theta_1 & \theta_2\end{bmatrix}$. The set of all possible angles for our two joints forms a two-dimensional space called *configuration space*. Obviously, for more complex skeletons with more degrees of freedom per joint, configuration space becomes multidimensional, but the concepts described here work equally well no matter how many dimensions we have.

Now imagine plotting a three-dimensional graph, where for each combination of joint rotations (i.e., for each point in our two-dimensional configuration space), we plot the distance from the end effector to the desired target. An example of this kind of plot is shown in Figure 12.45. The "valleys" in this three-dimensional surface represent regions in which the end effector is as close as possible to the target. When the height of the surface is zero, the end effector has reached its target. Inverse kinematics, then, attempts to find

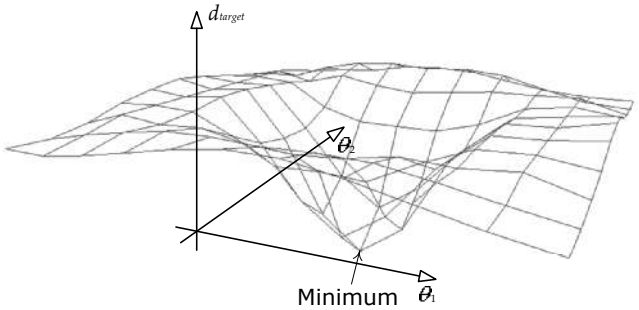

Figure 12.45. A three-dimensional plot of the distance from the end effector to the target for each point in two-dimensional configuration space. IK finds the local minimum.

minima (low points) on this surface.

We won't get into the details of solving the IK minimization problem here. You can read more about IK at http://en.wikipedia.org/wiki/Inverse_kinematics and in Jason Weber's article, "Constrained Inverse Kinematics" [47].

12.7.3 Rag Dolls

A character's body goes limp when he dies or becomes unconscious. In such situations, we want the body to react in a physically realistic way with its surroundings. To do this, we can use a *rag doll*. A rag doll is a collection of physically simulated rigid bodies, each one representing a semi-rigid part of the character's body, such as his lower arm or his upper leg. The rigid bodies are constrained to one another at the joints of the character in such a way as to produce natural-looking "lifeless" body movement. The positions and orientations of the rigid bodies are determined by the physics system and are then used to drive the positions and orientations of certain key joints in the character's skeleton. The transfer of data from the physics system to the skeleton is typically done as a post-processing step.

To really understand rag doll physics, we must first have an understanding of how the collision and physics systems work. Rag dolls are covered in more detail in Sections 13.4.8.7 and 13.5.3.8.

12.8 Compression Techniques

Animation data can take up a lot of memory. A single joint pose might be composed of ten floating-point channels (three for translation, four for rotation and

up to three more for scale). Assuming each channel contains a 4-byte floating-point value, a one-second clip sampled at 30 samples per second would occupy 4 bytes $\times$ 10 channels $\times$ 30 samples/second = 1200 bytes per joint per second, or a data rate of about 1.17 KiB per joint per second. For a 100-joint skeleton (which is small by today's standards), an uncompressed animation clip would occupy 117 KiB per joint per second. If our game contained 1,000 seconds of animation (which is on the low side for a modern game), the entire dataset would occupy a whopping 114.4 MiB. That's quite a lot, considering that a PlayStation 3 has only 256 MiB of main RAM and 256 MiB of video RAM. Sure, the PS4 has 8 GiB of RAM. But even so—we would rather have much richer animations with a lot more variety than waste memory unnecessarily. Therefore, game engineers invest a significant amount of effort into compressing animation data in order to permit the maximum richness and variety of movement at the minimum memory cost.

12.8.1 Channel Omission

One simple way to reduce the size of an animation clip is to omit channels that are irrelevant. Many characters do not require nonuniform scaling, so the three scale channels can be reduced to a single uniform scale channel. In some games, the scale channel can actually be omitted altogether for all joints (except possibly the joints in the face). The bones of a humanoid character generally cannot stretch, so translation can often be omitted for all joints except the root, the facial joints and sometimes the collar bones. Finally, because quaternions are always normalized, we can store only three components per quat (e.g., x, y and z) and reconstruct the fourth component (e.g., w) at runtime.

As a further optimization, channels whose pose does not change over the course of the entire animation can be stored as a single sample at time $t = 0$ plus a single bit indicating that the channel is constant for all other values of t.

Channel omission can significantly reduce the size of an animation clip. A 100-joint character with no scale and no translation requires only 303 channels—three channels for the quaternions at each joint, plus three channels for the root joint's translation. Compare this to the 1,000 channels that would be required if all ten channels were included for all 100 joints.

12.8.2 Quantization

Another way to reduce the size of an animation is to reduce the size of each channel. A floating-point value is normally stored in 32-bit IEEE format. This format provides 23 bits of precision in the mantissa and an 8-bit exponent.

However, it's often not necessary to retain that kind of precision and range in an animation clip. When storing a quaternion, the channel values are guaranteed to lie in the range $[-1, 1]$. At a magnitude of 1, the exponent of a 32-bit IEEE float is zero, and 23 bits of precision give us accuracy down to the seventh decimal place. Experience shows that a quaternion can be encoded well with only 16 bits of precision, so we're really wasting 16 bits per channel if we store our quats using 32-bit floats.

Converting a 32-bit IEEE float into an n-bit integer representation is called *quantization*. There are actually two components to this operation: *Encoding* is the process of converting the original floating-point value to a quantized integer representation. *Decoding* is the process of recovering an approximation to the original floating-point value from the quantized integer. (We can only recover an *approximation* to the original data—quantization is a *lossy* compression method because it effectively reduces the number of bits of precision used to represent the value.)

To encode a floating-point value as an integer, we first divide the valid range of possible input values into N equally sized *intervals*. We then determine within which interval a particular floating-point value lies and represent that value by the *integer index* of its interval. To decode this quantized value, we simply convert the integer index into floating-point format and shift and scale it back into the original range. N is usually chosen to correspond to the range of possible integer values that can be represented by an n-bit integer. For example, if we're encoding a 32-bit floating-point value as a 16-bit integer, the number of intervals would be $N = 2^{16} = 65{,}536$.

Jonathan Blow wrote an excellent article on the topic of floating-point scalar quantization in the *Inner Product* column of Game Developer Magazine, available at https://bit.ly/2J92oiU. The article presents two ways to map a floating-point value to an interval during the encoding process: We can either *truncate* the float to the next lowest interval boundary (*T encoding*), or we can *round* the float to the center of the enclosing interval (*R encoding*). Likewise, it describes two approaches to reconstructing the floating-point value from its integer representation: We can either return the value of the *left-hand side* of the interval to which our original value was mapped (*L reconstruction*), or we can return the value of the center of the interval (*C reconstruction*). This gives us four possible encode/decode methods: TL, TC, RL and RC. Of these, TL and RC are to be avoided because they tend to remove or add energy to the dataset, which can often have disastrous effects. TC has the benefit of being the most efficient method in terms of bandwidth, but it suffers from a severe problem—there is no way to represent the value zero exactly. (If you encode 0.0f, it becomes a small positive value when decoded.) RL is therefore usually

the best choice and is the method we'll demonstrate here.

The article only talks about quantizing positive floating-point values, and in the examples, the input range is assumed to be $[0, 1]$ for simplicity. However, we can always shift and scale any floating-point range into the range $[0, 1]$. For example, the range of quaternion channels is $[-1, 1]$, but we can convert this to the range $[0, 1]$ by adding one and then dividing by two.

The following pair of routines encode and decode an input floating-point value lying in the range $[0, 1]$ into an n-bit integer, according to Jonathan Blow's RL method. The quantized value is always returned as a 32-bit unsigned integer (`U32`), but only the least-significant n bits are actually used, as specified by the `nBits` argument. For example, if you pass `nBits==16`, you can safely cast the result to a `U16`.

```
U32 CompressUnitFloatRL(F32 unitFloat, U32 nBits)
{
    // Determine the number of intervals based on the
    // number of output bits we've been asked to produce.
    U32 nIntervals = 1u << nBits;

    // Scale the input value from the range [0, 1] into
    // the range [0, nIntervals - 1]. We subtract one
    // interval because we want the largest output value
    // to fit into nBits bits.
    F32 scaled = unitFloat * (F32)(nIntervals - 1u);

    // Finally, round to the nearest interval center. We
    // do this by adding 0.5f and then truncating to the
    // next-lowest interval index (by casting to U32).
    U32 rounded = (U32)(scaled + 0.5f);

    // Guard against invalid input values.
    if (rounded > nIntervals - 1u)
    rounded = nIntervals - 1u;

    return rounded;
}

F32 DecompressUnitFloatRL(U32 quantized, U32 nBits)
{
    // Determine the number of intervals based on the
    // number of bits we used when we encoded the value.
    U32 nIntervals = 1u << nBits;

    // Decode by simply converting the U32 to an F32, and
    // scaling by the interval size.
```

```
    F32 intervalSize = 1.0f / (F32)(nIntervals - 1u);
    F32 approxUnitFloat = (F32)quantized * intervalSize;

    return approxUnitFloat;
}
```

To handle arbitrary input values in the range $[min, max]$, we can use these routines:

```
U32 CompressFloatRL(F32 value, F32 min, F32 max,
                    U32 nBits)
{
    F32 unitFloat = (value - min) / (max - min);
    U32 quantized = CompressUnitFloatRL(unitFloat,
                                        nBits);
    return quantized;
}

F32 DecompressFloatRL(U32 quantized, F32 min, F32 max,
                      U32 nBits)
{
    F32 unitFloat = DecompressUnitFloatRL(quantized,
                                          nBits);
    F32 value = min + (unitFloat * (max - min));
    return value;
}
```

Let's return to our original problem of animation channel compression. To compress and decompress a quaternion's four components into 16 bits per channel, we simply call `CompressFloatRL()` and `DecompressFloatRL()` with $min = -1$, $max = 1$ and $n = 16$:

```
inline U16 CompressRotationChannel(F32 qx)
{
    return (U16)CompressFloatRL(qx, -1.0f, 1.0f, 16u);
}

inline F32 DecompressRotationChannel(U16 qx)
{
    return DecompressFloatRL((U32)qx, -1.0f, 1.0f, 16u);
}
```

Compression of translation channels is a bit trickier than rotations, because unlike quaternion channels, the range of a translation channel could theoretically be unbounded. Thankfully, the joints of a character don't move very far in practice, so we can decide upon a reasonable range of motion and flag an

error if we ever see an animation that contains translations outside the valid range. In-game cinematics are an exception to this rule—when an IGC is animated in world space, the translations of the characters' root joints can grow very large. To address this, we can select the range of valid translations on a per-animation or per-joint basis, depending on the maximum translations actually achieved within each clip. Because the data range might differ from animation to animation, or from joint to joint, we must store the range with the compressed clip data. This will add a tiny amount of data to each animation clip, but the impact is generally negligible.

```
// We'll use a 2 m range -- your mileage may vary.
F32 MAX_TRANSLATION = 2.0f;

inline U16 CompressTranslationChannel(F32 vx)
{
    // Clamp to valid range...
    if (vx < -MAX_TRANSLATION)
        vx = -MAX_TRANSLATION;
    if (vx > MAX_TRANSLATION)
        vx = MAX_TRANSLATION;

    return (U16)CompressFloatRL(vx,
                -MAX_TRANSLATION, MAX_TRANSLATION, 16);
}

inline F32 DecompressTranslationChannel(U16 vx)
{
    return DecompressFloatRL((U32)vx,
               -MAX_TRANSLATION, MAX_TRANSLATION, 16);
}
```

12.8.3 Sampling Frequency and Key Omission

Animation data tends to be large for three reasons: first, because the pose of each joint can contain upwards of ten channels of floating-point data; second, because a skeleton contains a large number of joints (250 or more for a humanoid character on PS3 or Xbox 360, and more than 800 on some PS4 and Xbox One games); third, because the pose of the character is typically sampled at a high rate (e.g., 30 frames per second). We've seen some ways to address the first problem. We can't really reduce the number of joints for our high-resolution characters, so we're stuck with the second problem. To attack the third problem, we can do two things:

- *Reduce the sample rate overall*. Some animations look fine when exported

at 15 samples per second, and doing so cuts the animation data size in half.

- *Omit some of the samples*. If a channel's data varies in an approximately linear fashion during some interval of time within the clip, we can omit all of the samples in this interval except the endpoints. Then, at runtime, we can use linear interpolation to recover the dropped samples.

The latter technique is a bit involved, and it requires us to store information about the *time* of each sample. This additional data can erode the savings we achieved by omitting samples in the first place. However, some game engines have used this technique successfully.

12.8.4 Curve-Based Compression

One of the most powerful, easiest-to-use and best-thought-out animation APIs I've ever worked with is Granny, by Rad Game Tools. Granny stores animations not as a regularly spaced sequence of pose samples but as a collection of *n*th-order, nonuniform, nonrational B-splines, describing the paths of a joint's S, Q and T channels over time. Using B-splines allows channels with a lot of curvature to be encoded using only a few data points.

Granny exports an animation by sampling the joint poses at regular intervals, much like traditional animation data. For each channel, Granny then fits a set of B-splines to the sampled dataset to within a user-specified tolerance. The end result is an animation clip that is usually significantly smaller than its uniformly sampled, linearly interpolated counterpart. This process is illustrated in Figure 12.46.

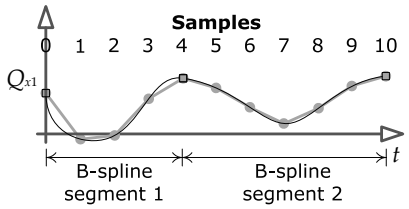

Figure 12.46. One form of animation compression fits B-splines to the animation channel data.

12.8.5 Wavelet Compression

Another way to compress animation data is to apply *signal processing theory* to the problem, via a technique known as *wavelet compression*. A wavelet is a function whose amplitude oscillates like a wave but whose duration is very

short, like a brief ripple in a pond. Wavelet functions are carefully crafted to give them desirable properties for use in signal processing.

In wavelet compression, an animation curve is decomposed into a sum of orthonormal wavelets, in much the same way that an arbitrary signal can be represented as a train of delta functions or a sum of sinusoids. We discuss signal processing and linear time-invariant systems in some depth in Section 14.2; the concepts presented there form the foundations necessary to understand wavelet compression. A full discussion of wavelet-based compression techniques is well beyond the scope of this book, but you can read more about it online. Search for "wavelet" to find introductory articles on the topic, and then try searching for "Animation Compression: Signal Processing" on Nicholas Frechette's blog for a great article on how wavelet compression was implemented for *Thief* (2014) by Eidos Montreal.

12.8.6 Selective Loading and Streaming

The cheapest animation clip is the one that isn't in memory at all. Most games don't need every animation clip to be in memory simultaneously. Some clips apply only to certain classes of character, so they needn't be loaded during levels in which that class of character is never encountered. Other clips apply to one-off moments in the game. These can be loaded or streamed into memory just before being needed and dumped from memory once they have played.

Most games load a core set of animation clips into memory when the game first boots and keep them there for the duration of the game. These include the player character's core move set and animations that apply to objects that reappear over and over throughout the game, such as weapons or power-ups. All other animations are usually loaded on an as-needed basis. Some game engines load animation clips individually, but many package them together into logical groups that can be loaded and unloaded as a unit.

12.9 The Animation Pipeline

The operations performed by the low-level animation engine form a *pipeline* that transforms its inputs (animation clips and blend specifications) into the desired outputs (local and global poses, plus a matrix palette for rendering).

For each animating character and object in the game, the animation pipeline takes one or more animation clips and corresponding blend factors as input, blends them together, and generates a single local skeletal pose as output. It also calculates a global pose for the skeleton and a palette of skinning

matrices for use by the rendering engine. Post-processing hooks are usually provided, which permit the local pose to be modified prior to final global pose and matrix palette generation. This is where inverse kinematics (IK), rag doll physics and other forms of procedural animation are applied to the skeleton.

The stages of this pipeline are:

1. *Clip decompression and pose extraction*. In this stage, each individual clip's data is decompressed, and a static pose is extracted for the time index in question. The output of this phase is a local skeletal pose for each input clip. This pose might contain information for every joint in the skeleton (a *full-body pose*), for only a subset of joints (a *partial pose*), or it might be a *difference pose* for use in additive blending.

2. *Pose blending*. In this stage, the input poses are combined via full-body LERP blending, partial-skeleton LERP blending and/or additive blending. The output of this stage is a single local pose for all joints in the skeleton. This stage is of course only executed when blending more than one animation clip together—otherwise the output pose from stage 1 can be used directly.

3. *Global pose generation*. In this stage, the skeletal hierarchy is walked, and local joint poses are concatenated in order to generate a global pose for the skeleton.

4. *Post-processing*. In this optional stage, the local and/or global poses of the skeleton can be modified prior to finalization of the pose. Post-processing is used for inverse kinematics, rag doll physics and other forms of procedural animation adjustment.

5. *Recalculation of global poses*. Many types of post-processing require global pose information as input but generate local poses as output. After such a post-processing step has run, we must recalculate the global pose from the modified local pose. Obviously, a post-processing operation that does not require global pose information can be done between stages 2 and 3, thus avoiding the need for global pose recalculation.

6. *Matrix palette generation*. Once the final global pose has been generated, each joint's global pose matrix is multiplied by the corresponding inverse bind pose matrix. The output of this stage is a palette of skinning matrices suitable for input to the rendering engine.

A typical animation pipeline is depicted in Figure 12.47.